

第六章 深度 Q 网络

回顾一下第三章的 Q 表格：是基于价值的方法；Q 表格记录了所有的 $Q(s, a)$ 的组合，根据 Q 表格可以做出每一步的最优决策。

但是，这只适用于状态数较少、离散状态的情况。如果**状态是连续的，状态的数量无穷大**，此时无法构建一个 Q 表来存下所有的 $Q(s, a)$ 的组合。这一章就解决这个问题。

此时我们想到可以用**一个深度神经网络来表示 Q 表格：输入状态，输出各动作的 Q**，即

$$Q_{\theta}(s, \cdot) = [Q_{\theta}(s, a_1), Q_{\theta}(s, a_2), \dots, Q_{\theta}(s, a_n)]$$

这就是深度 Q 网络 (DQN) 。

基于价值的方法的基础在于计算、预测价值。让我们回顾一下第三章里面我们是怎样预测并修正固定策略 π 下某状态 s 的价值 $V^{\pi}(s)$ 的：以蒙特卡洛方法为例，对于每个状态，我们从默认值开始，根据实际跑的结果进行更新：

$$V(s_t) \leftarrow V(s_t) + \alpha(G_t - V(s_t))$$

而现在的状态数非常大，不可能为每个状态都存储一个 $V(s)$ 。此时我们**引入神经网络 ϕ ：输入一个状态，经过神经网络 ϕ 处理得到当前状态的价值。**

这一点和第四章里面的用神经网络表示策略很像，只不过第四章是输入状态输出不同动作的概率，这里是输入状态输出当前状态的价值。

现在我们遇到一个问题：如何去更新价值？之前的情况很简单：知道当前状态价值 $V(s_t)$ ，跑蒙特卡洛得到 G_t ，然后直接用上面那个公式更新。

但是现在状态的价值是由神经网络 ϕ 生成的，我们要更新就**只能更新 ϕ 里面的参数**。所以这里就需要用到**监督学习**：预测值 = $V_{\phi}(s_t)$ ，目标值 = G_t 。然后定义**损失**：

$$L(\phi) = (V_{\phi}(s_t) - G_t)^2$$

然后反向传播、梯度下降来优化 ϕ 的参数实现更新。当然除了蒙特卡洛还可以换成 TD：只用把损失换成

$$L(\phi) = [V_{\phi}(s_t) - (R_{t+1} + \gamma V_{\phi}(s_{t+1}))]^2$$

目标网络

接下来我们的目标很明确：仿照我们第三章里面的 Q-learning，将现在连续状态情况的更新迁移到之前已有的经验上。第三章里面表格型 Q-learning 的更新公式为：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

同样道理，当 Q 是用神经网络计算出来的时候，我们无法像之前那样通过简单的数值加减来完成优化，所以需要用到监督学习：我们需要设立一个目标 y_t ，让当前神经网络的输出尽可能接近目标。一个自然的想法是：目标

$$y_t = r_{t+1} + \gamma \max_{a'} Q_{\theta}(s_{t+1}, a')$$

然后通过训练减小损失：

$$L(\theta) = [y_t - Q_{\theta}(s_t, a_t)]^2$$

但这里有一个很严重的问题：**目标和训练值 Q_{θ} 里面都使用了 θ 这个参数**，也就是说，一旦发生更新，目标和训练值都会一起改变，造成“自己追自己”，**不稳定**。

解决方法就是目标网络：**冻结住目标**中的 θ ，每训练一段时间将当前训练的 θ 与目标的 θ 同步。也就是说：

网络	记号	作用	是否每一步更新
在线网络 / 当前网络	Q_{θ}	计算当前预测值、选择动作	是
目标网络	$Q_{\bar{\theta}}$ 或 $\hat{Q}$	计算训练目标	暂时不更新

目标值改成：

$$y_t = r_{t+1} + \gamma \max_{a'} Q_{\bar{\theta}}(s_{t+1}, a')$$

当前预测值仍然由在线网络计算：

$$q_t = Q_{\theta}(s_t, a_t)$$

损失函数为：

$$L(\theta) = \left[r_{t+1} + \gamma \max_{a'} Q_{\bar{\theta}}(s_{t+1}, a') - Q_{\theta}(s_t, a_t) \right]^2$$

在这段时间内：

- Q_{θ} 每一步通过梯度下降更新；
- $Q_{\bar{\theta}}$ 保持不变；
- 因此目标 y_t 在一段时间内相对固定；
- 在线网络可以专心拟合这个目标。

需要强调：目标网络不是永远固定，而是**阶段性固定**。

一开始，两个网络的参数相同，然后开始循环：

1. 在线网络 Q_θ 计算当前预测值。
2. 目标网络 $Q_{\bar{\theta}}$ 计算训练目标。
3. **只更新在线网络参数 θ 。**
4. 重复若干次。
5. 将**在线网络参数复制给目标网络**。

注意：目标网络只能让训练**更稳定**，不能保证 Q 值没有估计误差。

探索

类似的，在 DQN 中也需要进行探索。除了之前的 ϵ -贪心以外，还有**玻尔兹曼探索**，也叫 softmax 探索，公式如下：

$$\pi(a | s) = \frac{\exp(Q(s, a)/T)}{\sum_{a'} \exp(Q(s, a')/T)}$$

其中 $T > 0$ 是温度系数。

它与 ϵ -贪心的区别是：它不是“要么完全贪心，要么完全随机”，而是根据 Q 值大小给每个动作分配概率。例如：

$$Q(s, a_1) = 2, \quad Q(s, a_2) = 5, \quad Q(s, a_3) = 3$$

那么 a_2 的概率最高， a_3 次之， a_1 最低，但三个动作都有机会被选择。

温度 T 越大，越趋向平均分布（偏向探索）；反之则更受到 Q 值差异本身的影响（Q 值大的动作被采用的几率大，偏向利用）。

经验回放

之前表格型 Q-learning 的过程是：采集一条信息，立刻用这条信息来更新 Q 表，然后进入下一个循环，这条信息只**被用一次就废弃**。在表格较小时没有大问题，但当使用神经网络时会遇到两个大困难：

1. 连续走的几步彼此之间**相关性高**，而传统的监督学习里各数据之间应当接近相互独立。
2. 一条经验只用一次**太浪费**。

因此，我们提出一个新的做法：**经验回放**。

建立一个缓冲区，存放已经得到的经验。训练时，从其中随机抽取若干条经验组成一个 batch 来更新 Q 网络；缓冲区满后，删除旧经验，写入新经验。

目标网络、探索、经验回放都是我们优化 DQN 的三个技巧，总结如下：

机制	解决的问题
ϵ -贪心 / 玻尔兹曼探索	避免只执行当前认为最好的动作
目标网络	避免训练目标变化过快
经验回放	避免数据高度相关，并重复利用历史经验

DQN 完整流程

伪代码：

初始化在线网络 Q_θ 。

初始化目标网络 $Q_{\bar{\theta}}$ ，并令 $\bar{\theta} \leftarrow \theta$ 。

初始化经验回放池 D 。

对于每个回合：

1. 重置环境，得到状态 s 。
2. 当回合未结束时，依次执行以下步骤：
 1. 使用 ϵ -greedy($Q_\theta(s, \cdot)$) 选择动作 a 。
 2. 执行动作 a ，得到 r 、 s' 、done。
 3. 将 $(s, a, r, s', \text{done})$ 存入 D 。
 4. 从 D 中随机采样一个批次，计算 $y = r + \gamma(1 - \text{done}) \max_{a'} Q_{\bar{\theta}}(s', a')$ ，计算 $q = Q_\theta(s, a)$ ，计算损失 $L = \text{mean}((q - y)^2)$ ，只更新在线网络参数 θ 。
 5. 每隔 C 次更新，令 $\bar{\theta} \leftarrow \theta$ 。
 6. 将当前状态更新为 $s \leftarrow s'$ 。

注意：上面的 done 是“是否为终止状态”的标志，是则为 1，非则为 0。

文字版：DQN 训练时先初始化在线网络 Q_θ 、目标网络 $Q_{\bar{\theta}}$ 和经验回放池 D ，并令 $\bar{\theta} \leftarrow \theta$ ；每个回合从初始状态 s_t 开始，按照 ϵ -greedy 策略选择动作，即以概率 ϵ 随机探索、以概率 $1 - \epsilon$ 选择 $a_t = \arg \max_a Q_\theta(s_t, a)$ ，与环境交互后得到奖励 r_t 、下一状态 s_{t+1} 和终止标志 d_t ，将转移样本 $(s_t, a_t, r_t, s_{t+1}, d_t)$ 存入经验回放池；当样本足够时，从 D 中随机采样一个批次，用目标网络计算 TD 目标 $y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\bar{\theta}}(s'_i, a')$ ，再用在线网络计算实际动作的当

前估计 $q_i = Q_\theta(s_i, a_i)$ ，通过损失函数 $L(\theta) = \frac{1}{B} \sum_i (q_i - y_i)^2$ 反向传播更新在线网络参数 θ ；在线网络经过固定次数 C 的更新后，将其参数复制给目标网络，即 $\bar{\theta} \leftarrow \theta$ ，然后继续重复“选择动作—收集经验—随机采样—计算目标—更新在线网络—同步目标网络”的过程，直到训练结束，评估时令 $\varepsilon = 0$ ，始终选择 $\arg \max_a Q_\theta(s, a)$ 的贪心动作。整个过程本质上是把表格型 Q-learning 中的 $Q(s, a)$ 表格更新，替换成神经网络回归，并加入经验回放和目标网络来提高训练稳定性。